

ASSIGNMENT

Build a Mathematical Agent using LangGraph

LangChain • Google Gemini • Tools • LangSmith

Difficulty: Intermediate Total: 10 Marks + 2 Bonus Marks

1. Objective

Build a simple AI Mathematical Agent using LangGraph and Google Gemini. The agent should understand a user's mathematical request, select the appropriate tool, call that tool, and return the final answer. Integrate LangSmith so you can observe and analyze execution traces.

2. Scenario

Build an AI-powered Mathematical Assistant capable of handling different mathematical operations. Each operation must be implemented as a separate dedicated tool.

- Addition
- Subtraction
- Multiplication
- Division

The LLM should decide which tool to use. The actual calculation must be performed by the selected Python tool.

3. Required Architecture

```
User
|
v
LangGraph Agent
|
+-----+-----+
| | |
v v v
Addition Subtraction Multiplication
Tool Tool Tool
|
v
Division Tool
|
v
Tool Result
|
v
Final Answer
|
v
LangSmith Trace
```

4. Part 1 — Create Mathematical Tools

Addition Tool

Accept two numbers and return their sum.

```
10, 20 → 30
```

Subtraction Tool

Accept two numbers and return their difference.

```
50, 20 → 30
```

Multiplication Tool

Accept two numbers and return their product.

```
10, 5 → 50
```

Division Tool

Accept two numbers and return the result of division.

```
100, 4 → 25
```

Important: Do not create one generic calculator tool for all four operations.

5. Part 2 — Create the LangGraph Agent

Use LangGraph's `create_agent()` method. Use the Google Gemini model introduced in class and provide the four mathematical tools to the agent.

```
Gemini Model
+
System Prompt
+
Mathematical Tools
|
v
LangGraph Agent
```

6. Part 3 — System Prompt

Write your own system prompt. It should instruct the agent to:

- Act as a mathematical assistant.
- Use the appropriate tool for every calculation.
- Never perform a calculation itself when a suitable tool is available.
- Select the correct tool based on the user's request.
- Provide the final result clearly.
- Handle division by zero appropriately.
- Be concise.

Example guidance (write your own prompt; do not simply copy this):

```
You are a mathematical assistant.

Use the available tools for addition, subtraction,
multiplication, and division.

Always use the appropriate tool to perform calculations.
Do not calculate the result yourself when a suitable
tool is available.

Handle division by zero gracefully and provide a clear
final answer.
```

7. Part 4 — Test the Agent

Test 1 — Addition

Input: What is $125 + 375$?
Expected: 500

Test 2 — Subtraction

Input: Calculate $1000 - 275$.
Expected: 725

Test 3 — Multiplication

Input: What is 25 multiplied by 12?
Expected: 300

Test 4 — Division

Input: Calculate 144 divided by 12.
Expected: 12

Test 5 — Natural Language

Input: I have 25 apples and buy another 17. How many apples do I have?
Expected: 42

Test 6 — Different Wording

Input: What is the product of 15 and 8?
Expected: 120

Test 7 — Division by Zero

Input: What is 100 divided by 0?
Expected: Handle gracefully; application must not crash.

8. Part 5 — Integrate LangSmith

Configure LangSmith tracing for your application. Run the agent and inspect the traces in LangSmith.

```
User Request
↓
Agent / LLM
↓
Tool Selection
↓
Tool Call
↓
Tool Result
↓
Final LLM Response
```

For at least three different user queries, identify this execution flow.

9. Part 6 — Analyze the LangSmith Trace

- Which tool did the agent select?
- Why was that tool selected?
- What arguments were passed to the tool?
- What result did the tool return?
- What final response was generated by the LLM?
- How many LLM calls were made during execution?

Use the LangSmith trace to answer these questions. Do not guess.

10. Bonus Challenge ■

Add two additional mathematical tools:

- Modulus — $17 \% 5 \rightarrow 2$
- Power — 2 raised to the power of 5 $\rightarrow 32$

The agent should automatically select these tools when required.

11. Bonus Challenge ■■

Add a Square Root tool.

```
Input: What is the square root of 144?  
Expected result: 12
```

Update the system prompt so the agent knows about this capability.

12. Restrictions

You MUST use:

- Python
- LangGraph
- `create_agent()`
- Google Gemini
- Separate tools for mathematical operations
- LangSmith

You MUST NOT:

- Hard-code final answers.
- Perform calculations directly in the LLM prompt.
- Create one generic calculator tool for all operations.
- Use an external calculator API.
- Use RAG or a vector database.

13. Deliverables

- Source code (.py or .ipynb).
- Separate Addition, Subtraction, Multiplication, and Division tools.
- The system prompt used by your agent.
- Output/screenshots for at least 7 test cases.
- Screenshots of at least 3 LangSmith traces.
- Trace analysis for the 3 selected traces.

Trace Analysis Format

```
User Query:
```

```
Selected Tool:
```

Tool Arguments:

Tool Result:

Number of LLM Calls:

Final Response:

14. Questions for Students

1. Why should mathematical calculations be performed using tools instead of asking the LLM to calculate the result?
2. How does the agent decide which tool should be called?
3. What is the role of the system prompt?
4. What is the difference between an LLM response and a tool response?
5. What information can you observe in a LangSmith trace?
6. What happens when the user asks a question that cannot be handled by any available tool?
7. What happens if a tool raises an error?

15. Evaluation Criteria

Criteria	Marks
Mathematical tools implementation	2
LangGraph create_agent() implementation	2
Appropriate system prompt	1
Correct tool selection	2
Error handling	1
LangSmith integration & traces	1
Testing & documentation	1
Total	10

Bonus: Modulus & Power +1 mark; Square Root +1 mark.

16. Learning Outcome

```
USER
|
v
AGENT (LangGraph)
|
v
Tool Selection
|
+----> Addition
+----> Subtraction
+----> Multiplication
+----> Division
```

```
|  
v  
Tool Result  
|  
v  
Agent / LLM  
|  
v  
Final Answer  
|  
v  
LangSmith Trace
```

Key concept: The LLM decides what to do, the tool performs the actual operation, and LangSmith allows you to observe what happened during execution.